

Finding Money Launderers Using Heterogeneous Graph Neural Networks

Fredrik Johannessen^{*1} and Martin Jullum^{†2}

¹DNB, P.O. Box 1600, Sentrum, NO-0021 Oslo, Norway

²Norwegian Computing Center, P.O. Box 114, Blindern, NO-0314 Oslo, Norway

Abstract

Current anti-money laundering (AML) systems, predominantly rule-based, exhibit notable shortcomings in efficiently and precisely detecting instances of money laundering. As a result, there has been a recent surge toward exploring alternative approaches, particularly those utilizing machine learning. Since criminals often collaborate in their money laundering endeavors, accounting for diverse types of customer relations and links becomes crucial. In line with this, the present paper introduces a graph neural network (GNN) approach to identify money laundering activities within a large heterogeneous network constructed from real-world bank transactions and business role data belonging to DNB, Norway’s largest bank. Specifically, we extend the homogeneous GNN method known as the Message Passing Neural Network (MPNN) to operate effectively on a heterogeneous graph. As part of this procedure, we propose a novel method for aggregating messages across different edges of the graph. Our findings highlight the importance of using an appropriate GNN architecture when combining information in heterogeneous graphs. The performance results of our model demonstrate great potential in enhancing the quality of electronic surveillance systems employed by banks to detect instances of money laundering. To the best of our knowledge, this is the first published work applying GNN on a large real-world heterogeneous network for anti-money laundering purposes.

Keywords— graph neural networks, anti-money laundering, supervised learning, heterogeneous graphs, PyTorch Geometric.

1 Introduction

Money laundering is the activity of securing proceeds of a criminal act by concealing where the proceeds come from. The ultimate goal is to make it look like the proceeds originated from legitimate sources. Money laundering is a vast global problem, as it enables all types of crime where the goal is to make a profit. Because most money laundering goes undetected, it is difficult to quantify its effect on the global economy. However, a research report by United Nations – Office on Drugs and Crime (2011) estimates that 1-2 trillion US dollars are being laundered each year, which corresponds to 2-5% of global gross domestic product. Both national and international anti-money laundering (AML) laws regulate electronic surveillance and reporting of suspicious transaction activities in financial institutions. The purpose of the surveillance is to detect *suspicious activities* with a high probability of being related to money laundering, such that they can be manually investigated. The manual investigation is performed by experienced investigators, which inspect several aspects of the case, often involving multiple implicated customers, to then decide whether the behavior is suspicious enough to be reported to the authorities. See Section 2 for more details

^{*}fredjo89@gmail.com

[†]Corresponding author: jullum@nr.no

about this process. The present paper concerns the electronic surveillance process, which ought to identify a relatively small number of suspicious activities in a vast ocean of legitimate ones.

In the past few decades, the electronic surveillance systems in banks have typically consisted of several simple rules, created by domain experts, that use fixed thresholds and a moderate number of `if/else` statements that determine whether an alert is generated. Such rules are still in large parts what makes up the surveillance systems (Chen et al., 2018b). However, these rules fall short of providing efficiency (Fruth, 2018), for at least three key reasons: a) They rely on manual work to create and keep the rules up to date with the dynamically changing data (Chen et al., 2018b). This work increases with the complexity and number of rules. b) They are typically too simple to be able to detect money laundering with high precision, resulting in many low-quality alerts, i.e. *false positives*¹. c) Their simplicity makes them easy to circumvent for sophisticated money launderers, resulting in the possibility that severe crimes go undetected. The consequence of these three shortcomings is that banks spend huge resources on a very inefficient approach, and only a tiny fraction of illegal proceeds is being recovered (Pol, 2020).

Driven by multiple money laundering scandals in recent years², the shortcomings of the rule-based system are high on the agenda for regulators and financial institutions alike, and considerable resources are devoted to developing more effective electronic surveillance systems. One avenue that is explored is to use machine learning (ML) to automatically learn when to generate alerts (see e.g. Jullum et al. (2020), de Jesús Rocha-Salazar et al. (2021)). Compared to human capabilities, ML is superior at detecting complicated patterns in vast volumes of data. As a consequence, ML-based systems have the potential to provide detection systems with increased accuracy and the ability to identify more sophisticated ways of laundering money.

Money laundering is a social phenomenon, where groups of organized criminals often collaborate to launder their criminally obtained proceeds. In the networks relevant for AML, the nodes may consist of customers, while the edges between the nodes represent money transfers, shared address, or joint ownership, to name a few possibilities. Analyses of such networks can uncover circumstances that would be impossible to detect through a purely entity-based approach, simply because the necessary information would not be present. Therefore, ML methods that leverage these relational data have a substantial advantage over those that do not. The simplest way to incorporate network characteristics into ML methods is to create node features that capture information about the node’s role in the network, e.g. network centrality metrics or characteristics of its neighborhood. The features can then be used in a downstream machine learning task. This approach is, however, suboptimal as the generated features might not be the ones most informative for the subsequent classification, and the full richness of the relational data will in any case not be passed over to the entity-based machine learning task.

Graph Neural Network (GNN) is a class of methods that overcome this drawback by applying the machine learning task directly on the network data through a neural network. GNNs are able to solve various graph-related tasks such as node classification (Kipf & Welling, 2016a), link prediction (Zhang & Chen, 2018), graph clustering (Wang et al., 2017), graph similarity (Bai et al., 2019) as well as unsupervised embedding (Kipf & Welling, 2016b). The primary idea behind GNNs is to extend the modern and successful techniques of *artificial neural networks* (ANNs) from regular tabular data to that of networks. In addition to their ability to incorporate both entity features and network features into a single, simultaneously trained model, most GNNs scale linearly with the number of edges in the network, making them applicable to large networks. A huge benefit of GNNs in practical use cases is that they are inductive rather than transductive: While transductive models (e.g. Node2Vec (Grover & Leskovec, 2016)) can only be used on the specific data that was present during training, inductive models can be applied to entirely new data without having to be retrained. This is crucial for the AML application where new transactions (edges) appear continuously, and customers (nodes) enter and leave on a daily basis.

Most GNNs are developed for *homogeneous* networks with a single type of entity (node) and a single type of relation (edge). In the present AML use case, the relevant network is *heterogeneous* in both nodes and edges: The nodes represent both private customers, companies, and external accounts, while the edges represent both financial transactions and professional business ties between the nodes. There exist some GNNs in the literature that are able to handle heterogeneous networks, such as: RGCN (Schlichtkrull et al.,

¹Hiring more workers to inspect each alert manually is often the resolution to compensate for this shortcoming.

²As an example, it was revealed in 2018 that the Estonian branch of Danske Bank, Denmark’s largest bank, carried out suspicious transactions for over €200 billion during 2007-2015 (Bjerregaard & Kirchmaier, 2019). Subsequent lawsuit claims have amounted to over €2 billion.

2018), HAN (Wang et al., 2019), MAGNN (Fu et al., 2020), HGT (Hu et al., 2020), and HetGNN (Zhang et al., 2019). A common issue with all these methods is that they are not designed to incorporate edge features. In our AML use case, this corresponds to properties of the financial transactions (or the business ties) and is crucial information for an effective learning task. Thus, based on our current knowledge and research, there exists no directly applicable GNN method for our AML use case.

The present paper proposes a heterogeneous GNN method that utilizes the edge features in the graph, which we denote *Heterogeneous Message Passing Neural Network* (HMPNN). The HMPNN method is an expansion of the MPNN method (Gilmer et al., 2017) to a heterogeneous setting. The extension essentially connects, and simultaneously trains multiple MPNN architectures, each working for different combinations of node and edge types. We investigate two distinct approaches to aggregate the embeddings of node-edge combinations in the final step: The first approach applies a simple summation of the embedding vectors derived from the various combinations. The second approach is novel and concatenates the embeddings before applying an additional single-layer neural network to enhance the aggregation process.

The HMPNN is developed for and applied to detect money laundering activities in a large heterogeneous network created from real-world bank transactions and business role data which belongs to Norway’s largest bank, DNB. The nodes represent bank customers and transaction counterparties, while the edges represent bank transactions and business ties. The network has a total of more than 5 million nodes and almost 10 million edges. Among the bank customers, some are known to conduct suspicious behavior related to money laundering. While the rest are assumed not fraudulent, there could be some undetected suspicious behavior also there. Thus, this is actually a semi-labeled dataset. Finally, no oversampling or undersampling was performed, which makes the class imbalance realistic to what one encounters in practical situations.

There are primarily two categories of bank customers: individual (retail) customers and organization (corporate) customers. Due to the fundamental differences between these two groups and their fraudulent behaviors, it is common practice to study and model their fraudulent behavior separately. In our setting, we, therefore, treat them as two distinct node types, each with its unique set of features. In this paper, we limit the scope to modeling, predicting, and detection of fraudulent *individual* customers. I.e. all fraudulent customers in our dataset are individual customers. We chose to restrict our analysis to individuals because there is a significantly larger number of known fraudulent individuals compared to organizations, providing the model with more fraudulent behavior to learn from. Furthermore, individuals are a more homogeneous group with simpler customer relationships compared to organizations, making them a more suitable group to apply our methodology to. Nevertheless, note that organizations may very well be involved and utilized by fraudulent individuals, and possess key connections in the graph that the model learns from.

In addition to network data, the nodes and edges have sets of features that depend on what type of node and edge they are. In this paper, HMPNN is compared to other state-of-the-art GNN methods and achieves superior results. To the best of our knowledge, there exists no prior published work on applying graph neural networks on a large real-world heterogeneous network for the purpose of AML.

The rest of this article is organized as follows: Section 2 provides some background and an overview of related work within the AML domain, as well as the GNN domain. In Section 3 we formulate our HMPNN model after introducing the necessary mathematical framework and notation. Section 4 presents the data for our AML use case, the setup of our experiment, and presents and discusses the results we obtain. Finally, Section 5 summarizes our contribution, and provides some concluding remarks and directions for further work. Additional model details are provided in AppendixA and B.

2 Background and related work

In this section, we provide some background on the AML process and give an overview of earlier, related work in the domain of AML. We also briefly describe the state-of-the-art of homogeneous and heterogeneous GNNs in the general case.

2.1 AML process

As mentioned in the Introduction, financial institutions are required by law to have effective AML systems in place. Figure 1 illustrates a typical AML process in a bank. The electronic surveillance system generates what is called *alerts* (1) on some of the transactions, or transaction patterns, that go through the bank.

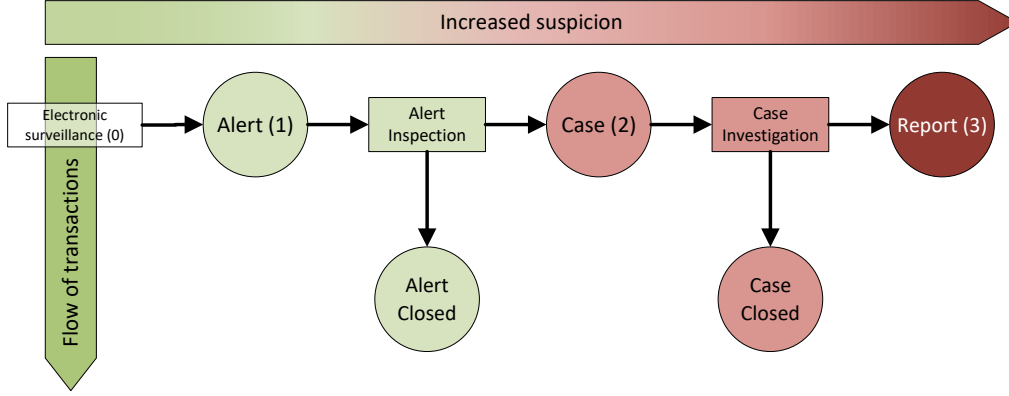


Figure 1: Workflow following detections from the electronic surveillance system.

An alert is first subject to a brief initial inspection, where those that can easily be identified as legitimate are picked out and marked as closed. Otherwise, the alert is upgraded to a *case* (2). At this stage, multiple alerts might be merged into a single case, which regularly involves multiple implicated customers. The case is then inspected thoroughly by experienced investigators. If money laundering is ruled out, the case will be marked as closed. Otherwise, the case will be upgraded to a *report* (3), and the revealed circumstances are reported in detail to the national *Financial Intelligence Unit* (FIU). From here on, the FIU oversees further action and will determine whether to start a criminal investigation.

The manual investigation part of the process is difficult to set aside for two reasons: a) The process is hard to automate, as the investigators sit on crucial, yearlong experience and often use non-quantifiable information sources to build their cases. b) AML laws do typically not allow automated reporting of suspicious behavior. The electronic surveillance generating the alerts is suitable for automation, and the benefits of better and more targeted alerts with fewer false positives, are huge for financial institutions. That is also why this paper is concerned with the electronic surveillance aspect of the AML process.

2.2 AML literature

In AML literature, there are very few papers that have datasets with real money laundering cases, and the majority of articles are validated on small datasets consisting of less than 10,000 observations (Chen et al., 2018b).

Both supervised (Liu et al. (2008), Deng et al. (2009), Zhang & Trubey (2019), Jullum et al. (2020)) and unsupervised (Lorenz et al. (2020), de Jesús Rocha-Salazar et al. (2021)) machine learning have been applied to AML use cases. However, the literature is quite limited, particularly for papers utilizing relational data.

Some AML papers apply the aforementioned strategy of generating network features used in a downstream machine learning task: Savage et al. (2016) create a graph from cash- and international transactions reported to the Australian FIU (AUSTRAC), from which they extract communities defined by filtered k -step neighborhoods around each node, to then create summarizing features used to classify community suspiciousness using supervised learning. Colladon & Remondi (2017) construct multiple graphs (each of which aims to reveal unique aspects) from customer and transaction data belonging to an Italian factoring company. On these graphs, they report significant correlation between traditional centrality metrics (e.g. betweenness centrality) and known high-risk entities. Elliott et al. (2019) uses a combination of network comparison and spectral analysis to create features that are applied in a downstream learning algorithm to classify anomalies.

There are only a few papers applying GNNs to money laundering detection: In a brief paper, Weber et al. (2018) discuss the use of GNN on the AML use case. The paper provides some initial results of the scalability of GCN (Kipf & Welling, 2016a) and FastGCN (Chen et al., 2018a) for a synthetic data set. However, no results on the performance of the methods were provided. Weber et al. (2019) compare GCN to other non-relational ML methods on a real-world graph generated from bitcoin transactions, with

203k nodes and 234k edges. The authors highlight the usefulness of the graph data and find that GCN outperforms logistic regression, but it is still outperformed by random forest. The dataset, called the Elliptic data, is also released with the paper and has later been utilized by several others: Alarab et al. (2020) apply GCN extended with linear layers and improve significantly on the performance of the GCN model in Weber et al. (2019). Lo et al. (2023) apply a self-supervised GNN approach to create node embedding subsequently used as input in a random forest model, and reported good performance. Others (Alarab & Prakoonwit, 2022; Pareja et al., 2020) exploited the temporal aspect of the graph to increase the performance on the same data set.

It is unknown to what extent results from synthetic or bitcoin transaction networks are transferable to the real-life application of transaction monitoring of bank transactions. Our graph is also about 25 times larger than the Elliptic dataset. Moreover, as we will review immediately below, much has happened in the field of GNN since the introduction of GCN in 2018. Finally, to the best of our knowledge, there are no papers applying GNN (or other methodology) to an AML use case with a *heterogeneous* graph.

2.3 General case GNN literature

The history of GNNs can be traced back about twenty years and GNNs have during the past few years surged in popularity. This was kicked off by Kipf & Welling (2016a) who introduced the popular method *Graph Convolutional Network* (GCN). For an excellent survey of GNNs including their history, we refer to Wu et al. (2020). The core dynamics in GNNs is an iterative approach where each node receives information from its neighbors, and combines it with its own representation to create a new representation, which will be forwarded to its neighbors in the next iteration. We call this *message passing*. After a few iterations, these node representations are used to make inference about the node.

Concentrating on today’s most popular group of GNNs, *Spatial-based convolutional GNNs*, we briefly review some relevant homogeneous and heterogeneous GNN methods below.

2.3.1 Homogeneous GNN literature

The GCN method (Kipf & Welling, 2016a) is motivated by spectral convolution and was originally formulated for the transductive setting operating directly on the adjacency matrix of the graph. However, the method can be reformulated in the inductive and spatial-based GNN setting using a message-passing formulation: The message received by a node from its neighbors is a weighted linear transformation of the neighboring node representations, followed by aggregating the result by taking their sum.

GraphSage (Hamilton et al., 2017) expands on GCN in two ways: It uses a) a neighborhood sampling strategy to increase efficiency, and b) a neutral network, a pooling layer, and an LSTM (Hochreiter & Schmidhuber, 1997) to aggregate the incoming messages instead of a simple sum. A drawback of GraphSage is, however, that it does not incorporate edge weights.

The Graph attention network (GAT) of Veličković et al. (2017) introduced the attention mechanism into the GNN framework. This mechanism learns the relative importance of a node’s neighbors, to assign more weight to the most important ones.

Gilmer et al. (2017) presented the Message Passing Neural Network (MPNN) framework, unifying a large group of different GNN models. Apart from the unifying framework, the most essential contribution of this model is in our view that it applies a learned message-passing function that utilizes edge features.

2.3.2 Heterogeneous GNN literature

Heterogeneous graph neural networks are commonly defined as extensions of existing homogeneous graph neural networks. For instance, the Relational Graph Convolution Network (RGCN) (Schlichtkrull et al., 2018) extends the GCN framework to support graphs with multiple types of edges. RGCN achieves this by breaking down the heterogeneous graph into multiple homogeneous ones, one for each edge type. In each layer, GCN is applied to each homogeneous graph, and the resulting node embeddings are element-wise summed to form the final output. A drawback of RGCN is that it does not take node heterogeneity into account. Heterogeneous Graph Attention Networks (HAN) (Wang et al., 2019) generalize the Graph Attention Network (GAT) approach to heterogeneous graphs by considering messages between nodes connected by so-called meta-paths. Meta-paths (see the formal definition in Definition 2) are composite relationships between nodes that help to capture the rich structural information of heterogeneous graphs. HAN defines

two sets of attention mechanisms. The first is between two different nodes, which is analogous to GAT. The second set of attention mechanisms is performed at the level of meta-paths, which computes the importance score of different composite relationships. Metapath Aggregated Graph Neural Network (MAGNN) (Fu et al., 2020) extends the approach of HAN by also considering the intermediary nodes along each meta-path (Dong et al., 2017). While HAN computes node-wise attention coefficients by only considering the features of the nodes at each end of the meta-path, MAGNN transforms all node features along the path into a single vector.

The interest in GNNs is rapidly growing, and advancements in this field are consistently being made. There are several other methods available that haven’t been discussed here. For a more comprehensive understanding, we once again refer to the survey conducted by Wu et al. (2020), which provides an overview of GNNs in general. Additionally, for insights specifically on Heterogeneous Network Representation Learning, including Heterogeneous GNNs, we refer to Yang et al. (2020) for a good overview.

3 Model

In this section we define and describe our proposed heterogeneous GNN model, which is based on the generic framework from Message Passing Neural Network (MPNN) introduced in Gilmer et al. (2017). We start by introducing the original (homogeneous) MPNN model and algorithm before we move on to give precise definitions for our heterogeneous graph setup and present our novel extension of the MPNN model for heterogeneous networks.

3.1 Message Passing Neural Network (MPNN)

Gilmer et al. (2017) introduces the generic MPNN framework which is able to express a large group of different GNN models, including GCN, GraphSage, and GAT. This is done by formulating the message passing with two learned functions, $M_k(\cdot)$, called the *message functions*, and $U_k(\cdot)$, called the *node updated functions*, with forms to be specified later. The framework runs K message passing iterations $k = 1, \dots, K$ between nodes along the edges that connect them. The node representation vectors are initialized as their feature vectors, $\mathbf{h}_v^0 = \mathbf{x}_v$, and the previous representation $\mathbf{h}_v^{(k-1)}$ is the message that is being sent in each iteration k . After K iterations, the final representation $\mathbf{h}_v^{(K)}$ is passed on to an output layer to perform e.g. node-level prediction tasks. The message-passing function is defined as

$$\begin{aligned} \mathbf{m}_v^{(k)} &= \sum_{u \in N(v)} M_k(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{r}_{uv}), \\ \mathbf{h}_v^{(k)} &= U_k(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(k)}), \end{aligned} \tag{1}$$

where $N(v)$ denotes the neighborhood of node v , and $\mathbf{r}_{uv}$ represents the edge features for the edge between node u and v . By defining specific forms of $U_k(\cdot)$ and $M_k(\cdot)$, a distinct GNN method is formulated. From the viewpoint of this paper, an essential attribute of the MPNN framework is that the learned message-passing function utilizes edge features. Not many other proposed GNNs incorporate edge features into their model. Gilmer et al. (2017) emphasize the importance of edge features in the dataset they experiment on. For our dataset, the edge features contain essential information related to transactions and are required to be utilized in an expressive model.

3.2 Formal definitions

Before we can formulate our *heterogeneous* MPNN framework, we need to establish a precise definition of a heterogeneous graph, as well as a couple of additional concepts.

We largely adopt the commonly used graph notation from Wu et al. (2020), and use a heterogeneous graph definition which is a slight modification to those in Yang et al. (2020) and Wang et al. (2019) in order to allow for multiple edges of different types between the same two nodes.

Definition 1. (Heterogeneous graph) A heterogeneous graph is represented as $G = (V, E, \mathbf{X}, \mathbf{R}, Q^V, Q^E, \phi)$ where each node $v \in V$ and each edge $e \in E$ has a type, and Q^V and Q^E denote finite sets of predefined node types and edge types, respectively. Each node $v \in V$ has a node type $\phi(v) = \nu \in Q^V$, where $\phi(\cdot)$ is a node

type mapping function. Further, for $\phi(v) = \nu$, v has features $\mathbf{x}_v^\nu \in \mathbf{X}^\nu$, where $\mathbf{X}^\nu = \{\mathbf{x}_v^\nu \mid v \in V, \phi(v) = \nu\}$ and $\mathbf{X} = \{\mathbf{X}^\nu \mid \nu \in Q^V\}$. The dimension and specifications of the node feature $\mathbf{x}_v^\nu$ may be different for different node types ν . Further, let us denote by e_{uv}^ε an edge of type $\varepsilon \in Q^E$ pointing from node u to v . Each edge e_{uv}^ε has features $\mathbf{r}_{uv}^\varepsilon \in \mathbf{R}^\varepsilon$, where $\mathbf{R}^\varepsilon = \{\mathbf{r}_{uv}^\varepsilon \mid u, v \in V\}$ and $\mathbf{R} = \{\mathbf{R}^\varepsilon \mid \varepsilon \in Q^E\}$. Just like for nodes, the edge features may have different dimensions for different edge types.

To formulate heterogeneous message passing, we will use the concept of *meta-paths*. Meta-paths are commonly used to extend methods from a homogeneous to a heterogeneous graph. For example, Dong et al. (2017) use meta-paths when introducing *metapath2vec*, which extends *DeepWalk* (Perozzi et al., 2014) and the closely related *node2vec* (Grover & Leskovec, 2016) to a method applicable on heterogeneous graphs. Wang et al. (2019) use meta-paths to generalize the approach of graph attention networks (Velićković et al., 2017) to that of heterogeneous graphs when formulating *heterogeneous graph attention network* (HAN). In addition to meta-path, the below definition introduces our own term, *meta-steps*, which we use when formulating our model.

Definition 2. (Meta-path, meta-step) A *Meta-path* belonging to a heterogeneous graph G is a sequence of specific edge types between specific node types,

$$(\nu_0, \varepsilon_1, \nu_1, \varepsilon_2, \dots, \nu_{k-1}, \varepsilon_k, \nu_k), \quad \nu_i \in Q^V, \varepsilon_i \in Q^E.$$

Here, k is the length of the meta-path. Let S be the set of meta-paths of length 1,

$$S = \{s = (\mu, \varepsilon, \nu) \mid \mu, \nu \in Q^V, \varepsilon \in Q^E\},$$

and refer to the elements $s \in S$ as *meta-steps*.

Finally, we introduce a definition of node neighborhood over a specific meta-step:

Definition 3. (Meta-step specific node neighborhood) Let $N_\mu^\varepsilon(v)$ be the set of nodes of type μ which is connected to node v by an edge of type ε pointing to v :

$$N_\mu^\varepsilon(v) = \{u \in V \mid \phi(u) = \mu, e_{uv}^\varepsilon \in E\}.$$

We call $N_\mu^\varepsilon(v)$ the (incoming) node neighborhood to node v with respect to the meta-step $s = (\mu, \varepsilon, \phi(v)) \in S$.

3.3 Heterogeneous MPNN

We are now ready to formulate our heterogeneous version of the MPNN method (HMPNN). The complete algorithm is provided in Algorithm 1.

Our approach for extending a homogeneous GNN to a heterogeneous one is, essentially, the same as used by Schlichtkrull et al. (2018), where they generalize GCN to the method *Relational Graph Convolutional Network* (RGCN) applicable on graphs with multiple edge types.

The algorithm performs (at each iteration) multiple MPNN message passing operations, one for each meta-step $s \in S$ in the graph. Each of these has its separate learned functions $M_k^s(\cdot) = M_k^{(\mu, \varepsilon, \nu)}(\cdot)$ and $U_k^s(\cdot) = U_k^{(\mu, \varepsilon, \nu)}(\cdot)$, which allows the method to learn the context of each meta-step, and also allow message passing between nodes and across edges with varying numbers of features. The intermediate output of this process is multiple representation vectors for each node. To reduce these to a single vector, they are aggregated by a learned aggregation function $A^\nu(\cdot)$ which is specific to each node type. In line with the generic formulation of MPNN we do not specify a specific form of the aggregation function in Algorithm 1. During the experiments, we have assessed two alternative options, which we will discuss shortly.

Our HMPNN model is implemented in Python, using the library PyTorch Geometric (PyG) (Fey & Lenssen, 2019) allowing for high-performance computing by utilizing massive parallelization through GPUs. Source code is available here: <https://github.com/fredjo89/heterogeneous-mpnn>

Algorithm 1 Heterogeneous MPNN

Initialize the representation of each node as its feature vector,

$$\mathbf{h}_v^{(0)} = \mathbf{x}_v^\nu, \forall v \in V \text{ of type } \nu = \phi(v).$$

for $k = 1, \dots, K$ **do**

▷ For each iteration

for $v \in V$ **do**

 ▷ For each node

for $\mu \in Q^V, \varepsilon \in Q^E$ such that $N_\mu^\varepsilon(v) \neq \emptyset$ **do**

 ▷ For each meta-step ending at $\phi(v)$

 Compute the new representation for node v of type $\nu = \phi(v)$, for the specific meta-step,

$$\mathbf{m}_v^{(\mu, \varepsilon, k)} = \sum_{u \in N_\mu^\varepsilon(v)} M_k^{(\mu, \varepsilon, \nu)}(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{r}_{uv}^\varepsilon),$$

$$\mathbf{h}_v^{(\mu, \varepsilon, k)} = U_k^{(\mu, \varepsilon, \nu)}(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(\mu, \varepsilon, k)}).$$

end for

 Aggregate the representations from the multiple meta-steps into a single new representation for that node,

$$\mathbf{h}_v^{(k)} = A_k^{(\nu)}(\{\mathbf{h}_v^{(\mu, \varepsilon, k)} \mid \mu \in Q^V, \varepsilon \in Q^E\})$$

end for

end for

3.4 Choosing specific forms of the functions

As our setup is very general, the algorithm in Algorithm 1 gives rise to a whole range of new heterogeneous GNN methods. By defining specific forms of $U_k^{(\mu, \varepsilon, \nu)}(\cdot)$, $M_k^{(\mu, \varepsilon, \nu)}(\cdot)$ and $A_k^{(\nu)}(\cdot)$, a distinct GNN method is formulated. Note that there is nothing that prevents us from choosing different forms of these functions for different meta-steps or iterations. However, for the AML use case in Section 4 we have limited the scope to a single form for each of the three functions, respectively. These are described in the following.

As message function, we use the same as Gilmer et al. (2017):

$$M_k^{(\mu, \varepsilon, \nu)}(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{r}_{uv}^\varepsilon) = g_k^{(\mu, \varepsilon, \nu)}(\mathbf{r}_{uv}^\varepsilon) \mathbf{h}_u^{(k-1)}.$$

Here, $g_k^{(\mu, \varepsilon, \nu)}(\cdot)$ is a single layer neural network which maps the edge feature vector $\mathbf{r}_{uv}^\varepsilon$ to a $d^v \times d^u$ matrix, where d^u , and d^v are the number of features for the sending and receiving node type, respectively.

As update function we use

$$U_k^{(\mu, \varepsilon, \nu)}(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(\mu, \varepsilon, \nu, k)}) = \sigma(\mathbf{m}_v^{(\mu, \varepsilon, k)} + \mathbf{B}_k^{(\mu, \varepsilon, \nu)} \mathbf{h}_v^{(k-1)}),$$

where $\mathbf{B}_k^{(\mu, \varepsilon, \nu)}$ is a matrix. Note that for a homogeneous graph, our choices of $M(\cdot)$ and $U(\cdot)$ are similar to those in Hamilton et al. (2017), except that the matrix applied in the message function is conditioned on the edge features rather than being the same across all edges.

For the aggregation function, we consider two alternatives. The first is to take the sum of the vectors from each meta-step before performing a nonlinear transformation, in the same fashion as (Schlichtkrull et al., 2018):

$$A_k^{(\nu)}(\{\mathbf{h}_v^{(\mu, \varepsilon, k)} \mid \mu \in Q^V, \varepsilon \in Q^E\}) = \sigma\left(\sum_{\mu \in Q^V, \varepsilon \in Q^E} \mathbf{h}_v^{(\mu, \varepsilon, k)}\right). \quad (2)$$

Here, $\sigma(\cdot)$ is the sigmoid function. In the second aggregation method, the vectors $\mathbf{h}_v^{(\mu, \varepsilon, k)}$ are concatenated into a single vector. A single-layer perceptron (neural network) is then applied to it, and outputs the new representation:

$$A_k^{(\nu)}(\{\mathbf{h}_v^{(\mu, \varepsilon, k)} \mid \mu \in Q^V, \varepsilon \in Q^E\}) = \sigma\left(W_k^{(\nu)} \parallel \sigma(\mathbf{h}_v^{(\mu, \varepsilon, k)})\right). \quad (3)$$

We denote the two resulting models *HMPNN-sum* and *HMPNN-ct*, respectively. Figure 2 illustrates the architectural extension of the homogeneous MPNN model to our HMPNN model (HMPNN-ct) as messages are passed to one of the node types, where (3) is used as aggregation function.

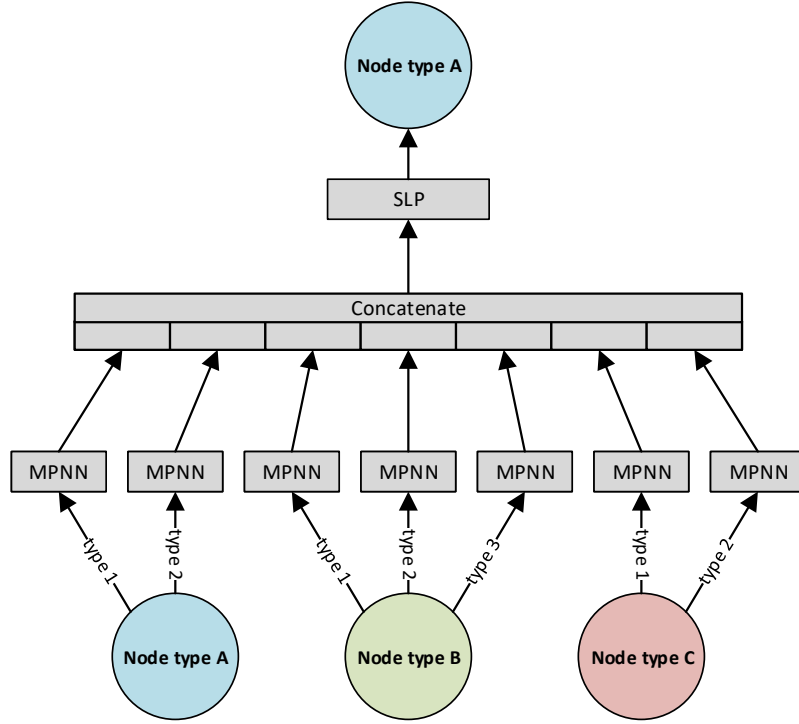


Figure 2: Illustration of the architectural extension of the homogeneous MPNN model to our HMPNN model (HMPNN-ct) as messages are passed to one of the node types (A), from three node types (A, B, and C) and with three edge types (1, 2 and 3). SLP is short for Single Layer Perceptron (neural network). Analogous architectures are used for messages passed to the other node types.

4 AML use case

In this section, we first describe the heterogeneous graph data. Then we lay out the setup of the experiments we have performed on this dataset before we provide the results.

4.1 Data

The graph is established based on customer and transaction data from Norway’s largest bank, DNB, in the period from February 1, 2022, to January 31, 2023. The nodes in the graph represent entities that are senders and/or recipients of financial transactions. If two entities participate in a transaction with each other, this is represented by an edge (of type transaction) between the two, where the direction of the edge points from the sender to the recipient. There are in total 5 million nodes and 9 million such edges in the graph.

There are three types of nodes in the graph. The first one is called *individual* and represents a human individual’s customer relationship in the bank. It includes all of the individual’s accounts in the bank³. The second type of node is called *organization*, and represents an organization’s or company’s customer relationship in the bank in the same manner as a node representing an individual. The third type of node is called *external* and represents a sender or recipient of a transaction that is outside of the bank.

The majority of the edges in the graph represent presence of a financial transaction between different individuals/organizations/external entities in the edge direction. In addition to this edge type, the graph includes role as a second edge type. That edge points from an individual to an organization if the individual

³Transactions made to/from any of the accounts in the bank belonging to the customer will result in an edge to/from the node representing the individual.

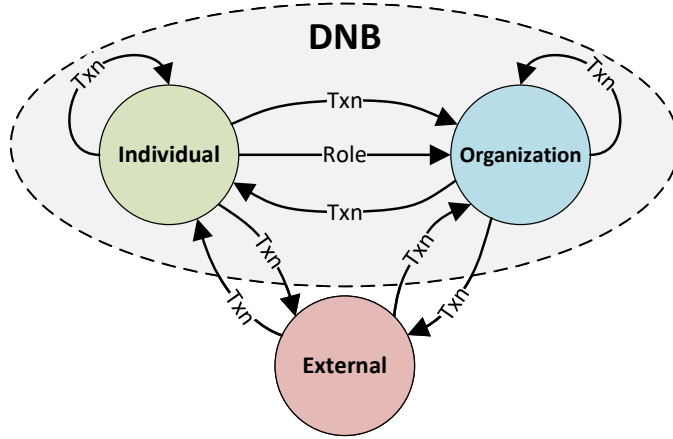


Figure 3: The schema of the graph that has been the subject of our experiments, with the nodes representing customer relationships in DNB outlined by the grey ellipse. Here, the transaction edges are abbreviated as *Txn*.

occupies a position on the board, is the CEO, or holds ownership in the organization. The resulting graph is directed and heterogeneous with respect to both nodes and edges. Figure 3 shows the schema of the graph, including the nine possible meta-steps. As shown in the schema, there are no edges between different external nodes, since the bank does not have access to transactions not involving their customers.

The nodes that represent individuals are assigned a binary class (0 for regular individuals, and 1 for individuals known to conduct suspicious behavior). As mentioned in the introduction, the data only contains labels for individual nodes. Suspicious individuals are defined as those that have been subject of an AML case (stage 2 in Figure 1) during a certain time window. Note that customers implicated in cases that were not reported to the FIU are still defined as suspicious. This decision was made because our objective is to model suspicious activity, which these customers certainly have conducted, even though the suspiciousness was diminished by a close manual inspection. Less than 0.5% of the individuals belong to class 1 (suspicious).

Due to the sensitive nature of these data, containing both personal and possibly competition sensitive information for the bank, the data are not shareable. We are neither allowed to reveal the exact details of the graphs nor the features associated with the different nodes/edges in our model. Below, we give a broad overview of the characteristics and features of the graph, within our permission restrictions. To get a feeling of the local characteristics of the graph, Figure 4 shows egonets of four random nodes, with the starting node enlarged. The upper and lower panels show, respectively, the 3-hop and 9-hop egonets of the (undirected) transaction and role edges. The number of shown hops was chosen to balance presentability and amount of detail. Moreover, Figure 5 shows histograms of the degree centrality for the three different node types. Some nodes have a large number of neighbors, while others have few. While the degree distribution of individuals and organizations is similar, the distribution for organizations has a thicker tail, indicating that it’s more common for organizations to exhibit a higher degree. As we don’t have knowledge of edges between external nodes, this node type typically has much fewer neighbors.

The three node types have separate sets of features that contain basic information about the entity. There are eleven, eight, and two node features for, respectively, individuals, organizations, and external nodes. The transaction edges have as features the number and monetary amount of transactions made within the one-year period. The role edges have two features, the first denoting the role type and the second the ownership percentage (provided the role represents ownership in the organization).

4.2 Experiment setup

The goal of this use case and experiment is to see to what extent our HMPNN method is able to predict the label (suspicious/regular) on nodes where the label is unknown to the model. As mentioned in the Introduction, we concentrate on building models for detecting fraudulent *individual* customers. This means

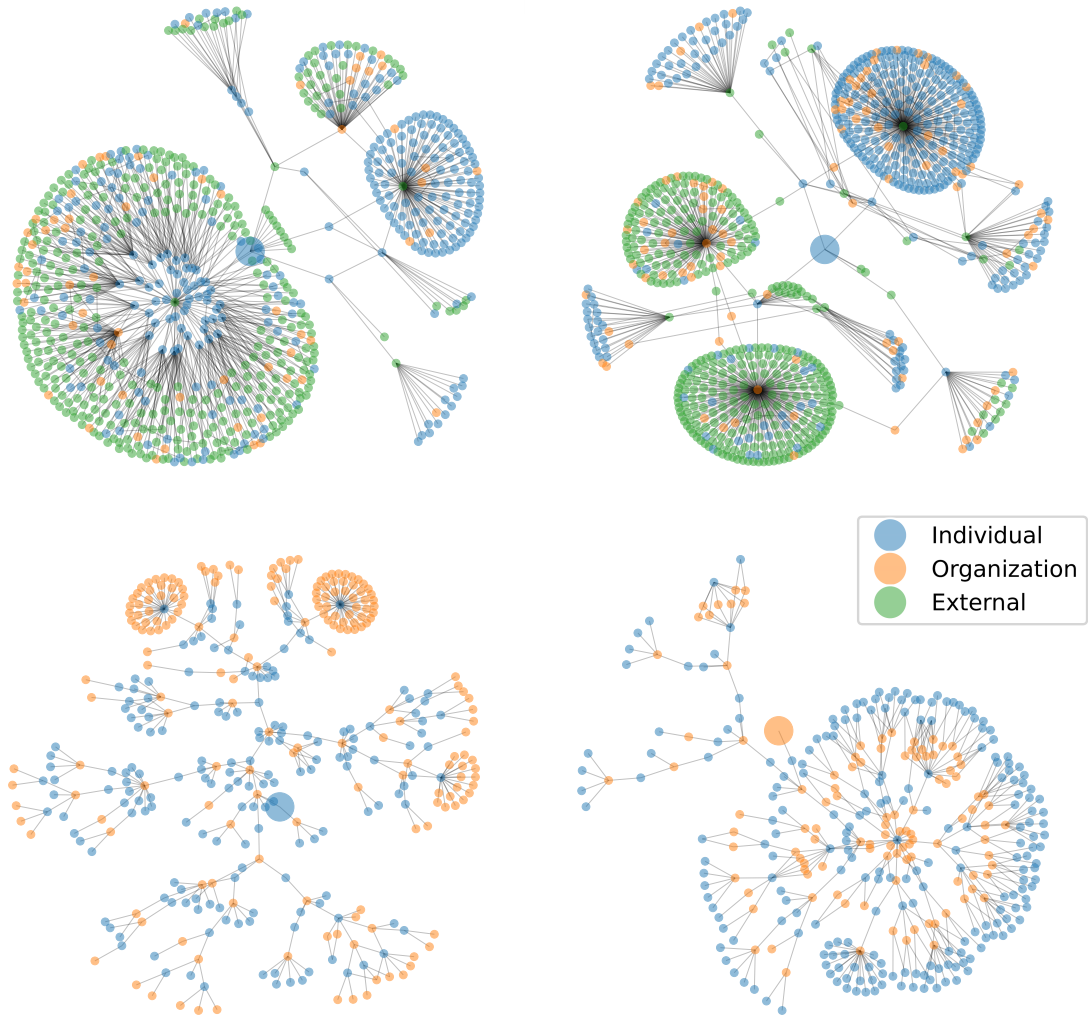


Figure 4: Homogeneous egonets for four random nodes in the graph. The upper two panels show 3-hop egonets of the (undirected) transaction edges in the graph. The lower two panels show 9-hop egonets for the (undirected) role edges in the graph. The starting node is enlarged.

that even if we use the entire graph for message passing, only individual nodes (in the training set) are assigned a label to learn from, and only individual nodes (in the test set) are subsequently predicted and evaluated.

To evaluate the performance of our methods, we benchmark and compare their performance to a set of alternative models/methods. To mimic a scenario with unknown labels, we split the nodes into a training set and a test set. The whole graph is available for each of the models when training them, but the labels are only available for the nodes in the training set. At testing time, each method attempts to predict the (unknown) nodes in the test set, and their performance is compared using different performance measures. We used a 70-30 train-test split, where the splitting was performed using stratified random sampling with allocation proportional to the original class balance, such that the class balance is preserved in the two sets. The same split was used for all methods.

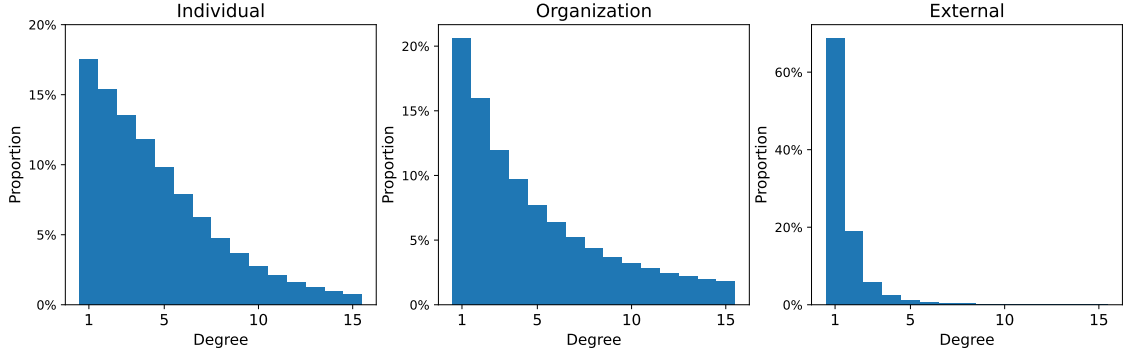


Figure 5: Histogram showing the distribution of the degree centrality for the three different node types, completely ignoring the edge type.

We compare the result of four model frameworks: two non-graph methods supplied with additional node features, and two GNN methods. The models are applied with multiple model complexity configurations. To enhance the non-graph methods, we generated 83 additional node features that accompany the original node features. These include 11 summaries of the node-neighborhoods, i.e. the number of neighbors of different types, both incoming and outgoing. We also computed 8 weighted summaries of node-neighborhoods, specifically for transaction edges and the monetary amount node feature. Additionally, we added 64 features generated using metapath2vec. These were assembled from embeddings of dimension 8 generated for each meta-path of length 2 starting and ending at a node of type individual. Together with the 11 intrinsic node features, this results in a total of 94 features. For more detailed information on these additional node features, please refer to Appendix A. The four methods are briefly described below:

Logistic regression This classic method serves as a very basic benchmark not directly utilizing the network information and with a basic and inflexible parametric form. The method has access to the additional network-generated node features.

Regular Neural Network This method, applied with both 1 and 2 hidden layers, is much more flexible than the logistic regression model, but neither utilizes the network directly. The method has access to the additional network-generated node features.

HGraphSage GraphSage (Hamilton et al., 2017) is a well-known homogeneous GNN method and is applied to our heterogeneous graph in the same fashion as HMPNN, as described below. In contrast to MPNN, GraphSage does not utilize edge features. We, therefore, test its performance both with and without additional node features that hold the weighted in/out degrees for the transaction edges, and the weight is the transaction amount on the edges. This results in 6 additional node features for nodes of type *individual* and *organization*, and 4 on nodes of type *external*. We test the method with the aggregation function in (2) (HMPNN-sum). We applied the method with both one, two, and three hidden layers.

HMPNN This GNN method, described in Section 3.3, is our extension of MPNN to heterogeneous graphs. We test the method with the two aggregation functions in (2) (HMPNN-sum) and (3) (HMPNN-ct). We applied the method with both one, two, and three hidden layers for each of the aggregation methods.

In total, our experiment contains 15 different models/method variants. The number of parameters involved in each of these is listed in Table 4 in the Appendix.

The logistic regression and Regular Neural Network models were trained using the open source deep learning library Pytorch (Paszke et al., 2019). HMPNN and HGraphSage were trained using the open source library Pytorch Geometric (PyG), which expands Pytorch with utilities for representing and training GNNs.

All models were trained with the Adam optimiser (Kingma & Ba, 2014), using the Binary Cross Entropy loss function:

$$\text{Loss}(\hat{y}_v, y_v) = y_v \cdot \log(\hat{y}_v) + (1 - y_v) \cdot \log(1 - \hat{y}_v).$$

The hyperparameters for each of the methods were tuned using 5-fold cross-validation on the training set. Here, three hyperparameters were determined: (1) The regularization strength, (2) the learning rate, and (3) the number of training iterations, i.e., by early stopping. For all methods, the learning rate was in the range $[10^{-4}, 10^{-1}]$. As for regularization, the L_2 -constraint was used, and was in the range $[10^{-8}, 10^{-1}]$. The optimal value for the L_2 constraint was highly dependent on the complexity of the model to be trained.

The experiments were carried out using *python 3.7.0*, with PyTorch 1.12.1 and PyG 2.2.0. The computer used to run the experiments had 8 CPUs of the type *High-frequency Intel Xeon E5-2686 v4 (Broadwell)* processors, with 61GB shared memory, and one GPU of type *NVIDIA Tesla V100* with 16GB memory. This GPU has 5,120 CUDA Cores and 640 Tensor Cores.

4.3 Results

For each node in the test set, all the different methods output a score between 0 and 1, reflecting the probability that the node is a suspicious customer. To measure the overall performance of different methods on the test set we rely on the area under the precision/recall curve (PR AUC) and the area under the receiver operator curve (ROC AUC). PR AUC computes the area under the curve obtained by plotting the *precision* ($TP/(TP+FP)$) as a function of the *recall* ($TP/(TP+FN)$), and PR ROC computes the area under the curve obtained by plotting the recall as a function of *False Positive Rate* ($FP/(FP+TN)$). Here $TP/FP/TN/FN$ represents the number of classified nodes which are, respectively, true positives, false positives, true negatives, and false negatives. Figure 6 displays the ROC AUC and PR AUC on the test set for all different methods and number of neural network layers used by the respective methods.

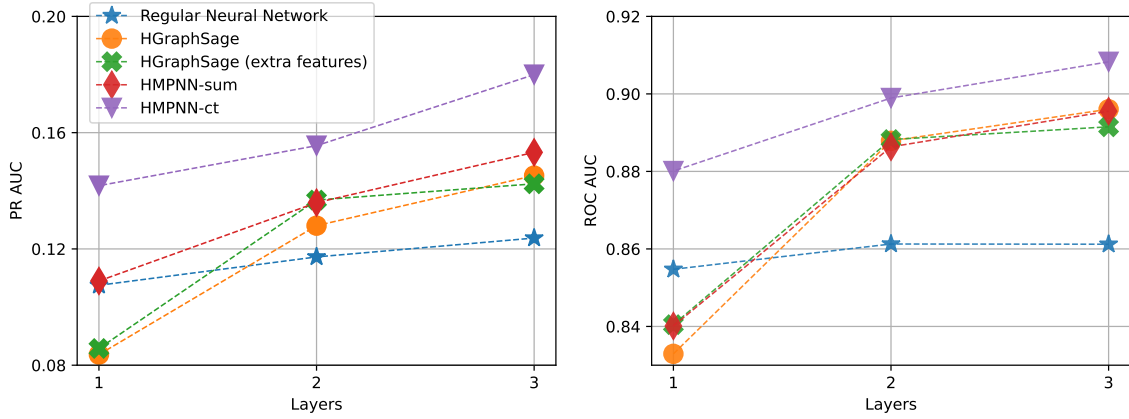


Figure 6: ROC AUC AND PR AUC on the test set for all different methods and number of neural network layers used by the methods. The Regular Neural Network with one layer corresponds to Logistic Regression.

All methods benefit from including more layers. HMPNN-ct with 3 layers is the best method in terms of both PR AUC and ROC AUC. While HMPNN-ct does quite well when applied with a single hidden layer, this is not the case for the other network models, at least when compared to the basic logistic regression (regular neural network with one layer). In terms of ROC AUC, the logistic regression model is actually better than all the other network models, and in terms of PR AUC, it is better than the HGraphSage models and comparable to HMPNN-sum. This is quite remarkable as the logistic regression model does not know anything about the network, except for additional network summary features, and has the simplest form of architecture. Note, however, that the Regular Neural Networks with 2 and 3 layers only do slightly better than Logistic Regression. This indicates that the simple architecture of the Logistic Regression model is not a significant downside for that limited data set. Moreover, the large performance gap to the HMPNN-ct model also shows that it is certainly possible to get more out of the network structure than the other network models manage. Thus, we believe that the lack of performance for the other network models

Model	Number of Layers	Recall(%)				PR AUC	ROC AUC
		1	5	10	50		
Regular Neural Network	1	61.54	36.28	28.55	5.53	0.1075	0.8547
	2	64.00	43.82	30.51	5.89	0.1173	0.8613
	3	61.54	47.56	34.68	5.88	0.1237	0.8612
HGraphSage	1	59.26	33.62	20.58	4.01	0.0836	0.8329
	2	61.54	48.15	38.56	6.51	0.1280	0.8879
	3	59.26	60.47	42.47	7.19	0.1452	0.8960
HGraphSage (extra features)	1	48.48	34.21	21.29	4.24	0.0858	0.8405
	2	64.00	50.65	35.39	7.41	0.1368	0.8882
	3	69.57	50.32	38.27	7.68	0.1424	0.8915
HMPNN-sum	1	64.00	38.24	29.75	5.34	0.1090	0.8401
	2	69.57	42.62	36.38	7.76	0.1359	0.8863
	3	84.21	53.79	38.27	8.67	0.1532	0.8955
HMPNN-ct	1	76.19	48.75	38.18	6.92	0.1418	0.8801
	2	61.54	50.65	43.66	8.96	0.1555	0.8989
	3	66.67	58.21	50.99	10.25	0.1800	0.9083

Table 1: Precision at specific values of Recall, in addition to PR AUC and ROC AUC for the different models

is related to an inappropriate and inefficient architecture compared to that of HMPNN-ct. The fact that overall, HMPNN-sum does not perform on par with HMPNN-ct further indicates that the performance boost in HMPNN-ct is mainly due to the architectural trick of the last single-layer neural network.

Considering the limitations in resources faced by banks, conducting thorough examinations of a substantial volume of suspicious cases is typically unfeasible. Therefore, the primary purpose of the model is to generate a limited set of high-quality predictions where money laundering is likely to occur, meaning that the precision at small to medium-sized recall levels is more relevant than the higher ones. Table 1 shows the precision corresponding to recall levels of 1%, 5%, 10% and 50%, respectively, and allows studying the performance of the methods in greater depth and at a wider range.

Focusing on HMPNN-ct, we see that when the classification threshold is set such that we identify 1% of the suspicious customers (recall = 1%) two-thirds of those classified as suspicious *are* actually suspicious (precision $\approx 67\%$). Increasing the classification threshold to 5% or 10% gives precisions of about 58% and 51%. Moreover, if we decrease the threshold such that half of the suspicious customers (recall = 50%) are identified, 90% of the customers classified as suspicious are not really suspicious. These rates may not seem impressive at first glance. Considering the severe class imbalance in the data (less than 0.5% of the total number of observations are suspicious), and the fact that detection of money laundering is a notoriously difficult problem, these performance scores are, actually, very promising.

Finally, note that even though the 3-layer HMPNN-sum model performs worse than HMPNN-ct overall and for the larger recalls, it obtains a significantly better precision (84% vs 67%) at recall 1%. In essence, this model is better at detecting the most evident instances of money laundering. This aspect is crucial to consider when selecting a model, particularly if resource limitations restrict the investigation to a small number of customers for potential money laundering.

5 Summary and concluding remarks

The present paper proposed and applied a heterogeneous extension of the homogeneous GNN model, MPNN (Gilmer et al., 2017), to detect money laundering in a large-scale real-world heterogeneous graph. The graph is derived from data originating from Norway’s largest bank, encompassing 5 million nodes and close to 10 million edges, and comprises customer data, transaction data, and business role data. Our heterogeneous MPNN model (HMPNN) incorporates distinct message-passing operators for each combination of node and edge types to account for the graph’s heterogeneity. Two versions of the model are proposed: HMPNN-sum and HMPNN-ct. Notably, HMPNN-ct used a novel strategy for constructing the final node embeddings,

as the embeddings from each node-edge operator were concatenated and fed into a final single-layer neural network. Overall, this version outperformed HMPNN-sum as well as all alternative models by a significant margin. HMPNN-sum was, however, the best model at recall = 1%, i.e. it was most accurate for the customers assigned the largest probabilities of being suspicious.

As we saw from the overall measures in Figure 6, all models performed the best when fitted using 3 hidden layers. We may have gotten even better performance if we increased the number of layers further. However, 3 layers is where we hit the memory roof on our GPU, so we were unable to explore this in practice. It is worth noting that the HMPNN-ct architecture is also clearly the most successful when we are restricting the number of layers to 2 or 1. This is relevant as larger networks, and/or less computational resources or training time available, may in other situations demand a reduction in the number of layers. From Table 4 in the Appendix, we also see that apart from the regular neural network model, HMPNN-ct has the fewest number of model parameters of the 3-layer models, indicating that it has an efficient architecture.

Customers labeled as “regular” may, in fact, be suspicious and potentially involved in money laundering – they just haven’t been controlled in the existing AML system and are therefore labeled as “regular”. This scenario holds true for practically all instances of money laundering modeling. Consequently, if a customer labeled as “regular” is assigned a high probability of being suspicious by the model, it is possible that the customer has been mislabeled. As a result, modeling test phases with a labeled test set, as outlined here, can also serve as a means to generate suggestions for customers who warrant further investigation into their past behaviors. In other words, the modeling approach allows for the identification of customers who may require re-examination based on the model’s predictions, even if they were initially labeled as “regular”.

When implementing a predictive model for suspicious transactions within a real AML system, several crucial decisions need to be made. One vital consideration involves determining the optimal stage in the process (see Figure 1) for applying the predictions: either preceding the alert inspection or preceding the case investigation. If the predictions are used prior to the alert inspection, it might be preferable to set a classification threshold with a higher recall. On the other hand, if the predictions are applied before the case investigation, it may be more appropriate to select a more stringent threshold, i.e. that has a lower recall. This would help minimize the allocation of investigation resources towards false positives, leading to greater efficiency.

In order to increase the performance of our money laundering modeling approach, some aspects become readily apparent. While our data set is rich in terms of the number of nodes and edges, contains network data from both financial transactions and professional roles, and has a large number of edge and node features, it can always be richer. In our setup, the transaction edges contain the number and total monetary amount made in the one-year period. This could be refined by also including the standard deviation, median, and other summary statistics like in Jullum et al. (2020). Moreover, provided high-quality data can be obtained, it would be valuable to include customer links using connections from social media platforms, geographical information such as shared address or phone numbers, or even family relations.

As mentioned, organizational customers are fewer in number and exhibit less homogeneity compared to individuals, rendering them less suitable for modeling compared to individuals. It still presents a natural candidate for further work to develop models that make predictions on the bank’s organization customers. However, to truly see the potential of such a model, we believe that it is essential to expand the dataset in time to include more fraudulent organizations to learn from and enrich the data with more organization-specific features.

A significant limitation that applies to our work, as well as most endeavors related to money laundering detection, is the restricted nature of the data, which is confined to customers within a single bank. Although our graph includes “external” customers from other banks, the transactions and professional role links between customers in external banks are unavailable. This is primarily due to banks being either unwilling or prohibited from merging their customer information with other banks, due to security regulations or competitive considerations. Surmounting these data-sharing challenges among prominent financial institutions would enable the modeling of a more comprehensive network of transactions and relationships, leaving fewer hiding places for money launderers. Nevertheless, the administration of such collaborative, analytical, and modeling systems demands substantial resources and investments, a process that may take years to accumulate.

In any case, to the best of our knowledge, no scientific work has been published regarding the utilization of heterogeneous graph neural networks in the context of detecting money laundering on a large-scale

real-world graph. This paper should be viewed as a first attempt to leverage heterogeneous GNN architecture within AML and has showcased promising outcomes. We envision that our paper will provide invaluable perspectives and directions for scholars and professionals engaged in money laundering modeling. Ultimately, we hope this contribution will aid in the continuous endeavors to combat money laundering.

Code availability

The implementation of our HMPNN model is available here: <https://github.com/fredjo89/heterogeneous-mpnn>

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Funding: This work was supported by the Norwegian Research Council [grant number 237718].

A Network features

This appendix provides details of how we created additional node features that capture information about a node’s role in the network. These were utilized in our entity-based models (and HGraphSage) for benchmarking in Section 4.

We generated a total of 83 additional features, which when combined with the original 11 intrinsic node features, resulted in a total of 94 features. These additional features are categorized into three distinct types: 1) Unweighted Neighborhood summary consisting of 11 features, 2) Weighted Neighborhood summary consisting of 8 features, and 3) Metapath2vec embeddings with a total of 64 features.

Unweighted Neighborhood Summary. The unweighted neighborhood summary features encapsulate the (unweighted) in/out degree for each meta-step that nodes of type individual are part of, see Figure 3. This amounts to seven features, six from transaction edges and one from role edges. Further, four summary features are added: 1) The total in-degree, which is the count of all incoming edges, irrespective of the meta-step, 2) The total out-degree, which is the count of all outgoing edges, irrespective of the meta-step, 3) The total degree, which is the count of all edges, irrespective of the meta-step or the direction, 4) The total count of meta-steps in which the node is involved. In total, this gives 11 additional node features.

Weighted Neighborhood Summary. The weighted neighborhood summary features contain the weighted in/out degree for each meta-step involving transaction edge. Here the edge feature representing the monetary amount was used as edge weight. This provides six features for a node of type individual. Further, two summary features are added: 1) The total weighted in-degree, 2) The total weighted out-degree. In total, this gives 8 additional node features.

Metapath2vec Embeddings. Metapath2Vec was used to generate embeddings for each of the four meta-paths shown in Table 2. These are all meta-paths of length 2 that start and end at a node of type individual. The dimension of the embedding for each meta-path was set to 8. The embeddings were added as features on both the node at the start and end of the meta-path. This amounts to 64 additional node features. We used the implementation of MetaPath2Vec in Pytorch Geometric. Table 3 lists the parameters used in the creation of the embeddings.

B Network parameters

Table 4 shows the number of parameters for each of the models discussed in Section 4.

ind	$\xrightarrow{\text{txn}}$	ind	$\xrightarrow{\text{txn}}$	ind
ind	$\xrightarrow{\text{txn}}$	org	$\xrightarrow{\text{txn}}$	ind
ind	$\xrightarrow{\text{txn}}$	ext	$\xrightarrow{\text{txn}}$	ind
ind	$\xrightarrow{\text{role}}$	org	$\xrightarrow{\text{txn}}$	ind

Table 2: Meta-paths for Individuals

Parameter	Value
<code>embedding_dim</code>	8
<code>walk_length</code>	20
<code>context_size</code>	10
<code>walks_per_node</code>	10
<code>num_negative_samples</code>	1

Table 3: Hyperparameters for the creation of MetaPath2Vec-embeddings.

Model	Number of Layers		
	1	2	3
NeuralNetwork	95	9,025	17,955
HGraphSage	189	3,999	7,809
HGraphSage (extra features)	329	13,045	25,761
HMPNN-sum	296	6,536	12,776
HMPNN-ct	3,071	4,487	6,303

Table 4: Number of model parameters for the different models

References

- Alarab, I., & Prakoonwit, S. (2022). Graph-based LSTM for anti-money laundering: Experimenting temporal graph convolutional network with bitcoin data. *Neural Processing Letters*, (pp. 1–19).
- Alarab, I., Prakoonwit, S., & Nacer, M. I. (2020). Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In *Proceedings of the 2020 5th international conference on machine learning technologies* (pp. 23–27).
- Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., & Wang, W. (2019). Simgnn: A neural network approach to fast graph similarity computation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining* (pp. 384–392).
- Bjerregaard, E., & Kirchmaier, T. (2019). The danske bank money laundering scandal: A case study. Available at SSRN 3446636, .
- Chen, J., Ma, T., & Xiao, C. (2018a). FastGCN: Fast learning with graph convolutional networks via importance sampling. In *International Conference on Learning Representations*. International Conference on Learning Representations, ICLR.
- Chen, Z., Van Khoa, L. D., Teoh, E. N., Nazir, A., Karuppiah, E. K., & Lam, K. S. (2018b). Machine learning techniques for anti-money laundering (aml) solutions in suspicious transaction detection: a review. *Knowledge and Information Systems*, 57, 245–285.
- Colladon, A. F., & Remondi, E. (2017). Using social network analysis to prevent money laundering. *Expert Systems with Applications*, 67, 49–58.
- Deng, X., Joseph, V. R., Sudjianto, A., & Wu, C. J. (2009). Active learning through sequential design, with applications to detection of money laundering. *Journal of the American Statistical Association*, 104, 969–981.
- Dong, Y., Chawla, N. V., & Swami, A. (2017). metapath2vec: Scalable representation learning for heterogeneous networks. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 135–144).
- Elliott, A., Cucuringu, M., Luaces, M. M., Reidy, P., & Reinert, G. (2019). Anomaly detection in networks with application to financial transaction networks. *arXiv preprint arXiv:1901.00402*, .
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, .
- Fruth, J. (2018). Anti-money laundering controls failing to detect terrorists, cartels, and sanctioned states. <https://www.reuters.com/article/bc-finreg-laundering-detecting-idUSKCN1GP2NV>. Accessed: 2022-07-14.
- Fu, X., Zhang, J., Meng, Z., & King, I. (2020). Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. *Proceedings of The Web Conference 2020*, .
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *International conference on machine learning* (pp. 1263–1272). PMLR.
- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 855–864).
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (pp. 1025–1035).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9, 1735–1780.
- Hu, Z., Dong, Y., Wang, K., & Sun, Y. (2020). Heterogeneous graph transformer. *Proceedings of The Web Conference 2020*, .
- de Jesús Rocha-Salazar, J., Segovia-Vargas, M. J., & del Mar Camacho-Miñano, M. (2021). Money laundering and terrorism financing detection using neural networks and an abnormality indicator. *Expert Systems with Applications*, 169, 114470.

- Jullum, M., Løland, A., Huseby, R. B., Ånonsen, G., & Lorentzen, J. (2020). Detecting money laundering transactions with machine learning. *Journal of Money Laundering Control*, .
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, .
- Kipf, T. N., & Welling, M. (2016a). Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, .
- Kipf, T. N., & Welling, M. (2016b). Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, .
- Liu, X., Zhang, P., & Zeng, D. (2008). Sequence matching for suspicious activity detection in anti-money laundering. In *International conference on intelligence and security informatics* (pp. 50–61). Springer.
- Lo, W. W., Kulatilleke, G. K., Sarhan, M., Layeghy, S., & Portmann, M. (2023). Inspection-l: self-supervised GNN node embeddings for money laundering detection in bitcoin. *Applied Intelligence*, (pp. 1–12).
- Lorenz, J., Silva, M. I., Aparício, D., Ascensão, J. T., & Bizarro, P. (2020). Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity. *arXiv preprint arXiv:2005.14635*, .
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., & Leiserson, C. (2020). Evolvegcn: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 5363–5370). volume 34.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 32* (pp. 8024–8035). Curran Associates, Inc.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 701–710).
- Pol, R. F. (2020). Anti-money laundering: The world’s least effective policy experiment? together, we can fix it. *Policy Design and Practice*, 3, 73–94.
- Savage, D., Wang, Q., Chou, P., Zhang, X., & Yu, X. (2016). Detection of money laundering groups using supervised learning in networks. *arXiv preprint arXiv:1608.00708*, .
- Schlichtkrull, M., Kipf, T. N., Bloem, P., Berg, R. v. d., Titov, I., & Welling, M. (2018). Modeling relational data with graph convolutional networks. In *European semantic web conference* (pp. 593–607). Springer.
- United Nations – Office on Drugs and Crime (2011). Estimating illicit financial flows resulting from drug trafficking and other transnational organized crime.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. *arXiv preprint arXiv:1710.10903*, .
- Wang, C., Pan, S., Long, G., Zhu, X., & Jiang, J. (2017). Mgae: Marginalized graph autoencoder for graph clustering. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management* (pp. 889–898).
- Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., & Yu, P. S. (2019). Heterogeneous graph attention network. In *The world wide web conference* (pp. 2022–2032).
- Weber, M., Chen, J., Suzumura, T., Pareja, A., Ma, T., Kanezashi, H., Kaler, T., Leiserson, C. E., & Schardl, T. B. (2018). Scalable graph learning for anti-money laundering: A first look. *arXiv preprint arXiv:1812.00076*, .
- Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., & Leiserson, C. E. (2019). Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, .

- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2020). A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems*, 32, 4–24.
- Yang, C., Xiao, Y., Zhang, Y., Sun, Y., & Han, J. (2020). Heterogeneous network representation learning: A unified framework with survey and benchmark. *IEEE Transactions on Knowledge and Data Engineering*, .
- Zhang, C., Song, D., Huang, C., Swami, A., & Chawla, N. (2019). Heterogeneous graph neural network. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, .
- Zhang, M., & Chen, Y. (2018). Link prediction based on graph neural networks. *Advances in neural information processing systems*, 31.
- Zhang, Y., & Trubey, P. (2019). Machine learning and sampling scheme: An empirical study of money laundering detection. *Computational Economics*, 54, 1043–1063.